

Tarea Labs API

Nathaly Daza

alejandradaza05@gmail.com – ndaza@uniempresarial.edu.co

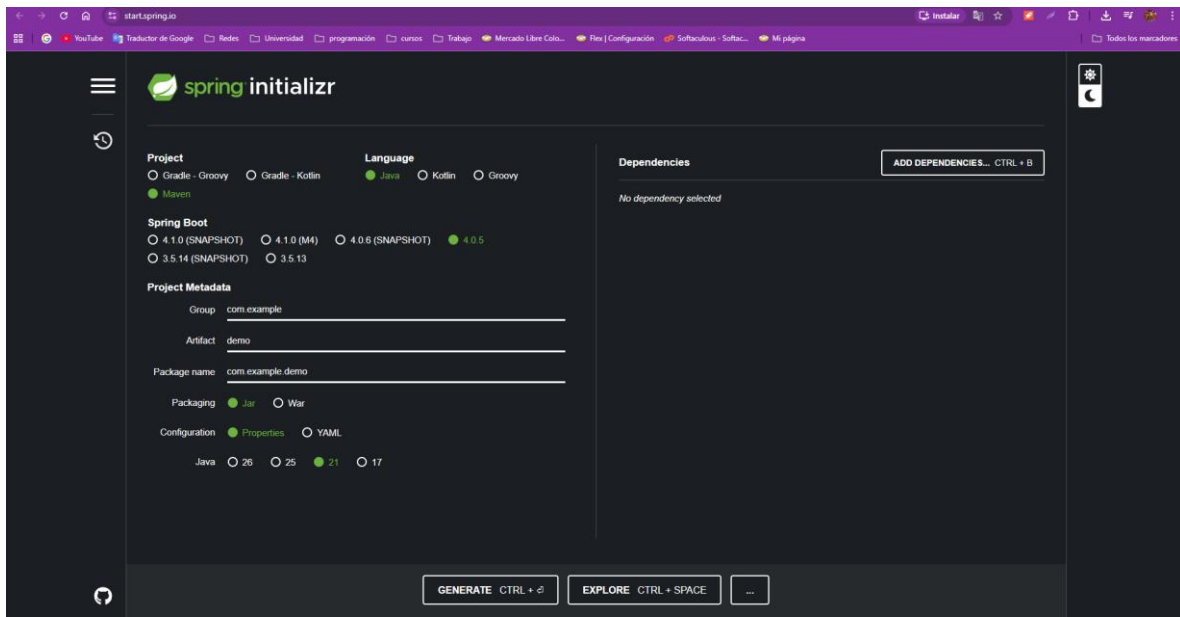
Tecnologías utilizadas	1
Arquitectura	2
Modelado del dominio	3
Control de concurrencia	4
Prueba concurrente	5
Respuestas a preguntas guía	9
Diagrama de arquitectura (capas)	10
Desarrollo	10

Este proyecto consiste en el desarrollo de una API REST para un sistema de comercio electrónico.

Permite gestionar usuarios, productos, inventario y órdenes de compra de forma segura, escalable y concurrente. El sistema fue diseñado para soportar múltiples usuarios realizando compras al mismo tiempo, garantizando que no se vendan más productos de los disponibles en inventario.

Tecnologías utilizadas

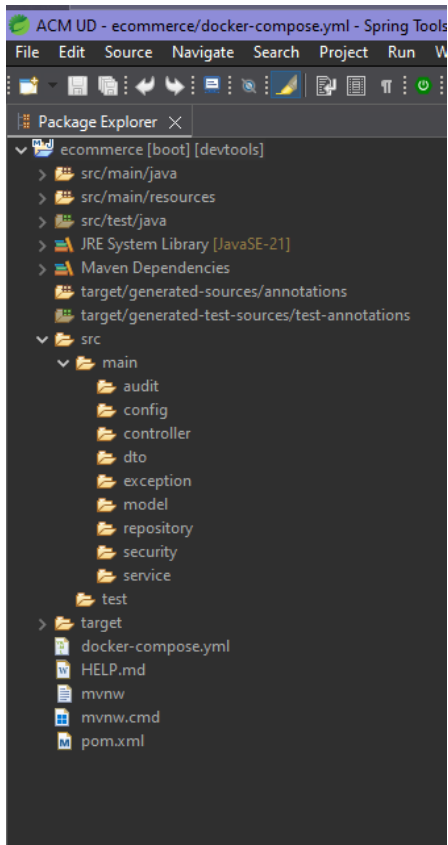
- Java 21
- Spring Boot
- Spring Security
- Spring Data JPA
- PostgreSQL (Docker)
- JWT (Json Web Token)
- Swagger / OpenAPI
- Lombok



Arquitectura

Se implementó una arquitectura en capas:

- Controller → Maneja las peticiones HTTP
- Service → Contiene la lógica de negocio
- Repository → Acceso a base de datos
- Model (Entity) → Representación de datos
- DTO → Transferencia de datos segura
- Security → Autenticación y autorización



Esta arquitectura permite separar responsabilidades, bajo acoplamiento, mayor mantenibilidad y facilidad de pruebas.

Modelado del dominio

- User
 - username
 - password
 - role (ADMIN / CLIENTE)
- Product
 - name
 - price
 - stock
 - version (control de concurrencia)
 - deleted (soft delete)
- Order
 - usuario
 - total
 - fecha
- OrderItem
 - producto
 - cantidad
 - precio

Se utilizó control de concurrencia optimista mediante:

```

1 package com.project.ecommerce.model;
2
3 import jakarta.persistence.*;
4
5
6 @Entity
7 @Data
8 public class Product {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Long id;
13
14     private String name;
15     private double price;
16     private int stock;
17
18     private boolean active = true;
19
20     @Version // Esto evita la sobreventa (concurrency optimista)
21     private Long version;
22
23
24     private boolean deleted = false; // Para el Soft Delete
25
26     @Column(updatable = false)
27     private java.time.LocalDateTime createdAt = java.time.LocalDateTime.now();
28
29 }

```

Cada vez que se actualiza un producto, su versión cambia

Si dos usuarios intentan comprar al mismo tiempo:

- Uno gana
- El otro recibe error de concurrencia

Se evita que el stock quede en valores negativos o inconsistentes.

Prueba concurrente

Se implementó un endpoint:

GET /test/concurrency

Este ejecuta múltiples compras simultáneamente usando hilos.

Algunas compras exitosas

Algunas fallidas por concurrencia o falta de stock

```
Order.java  OrderItem.java  Product.java  User.java  OrderItemRe...  OrderReposi...  TestC

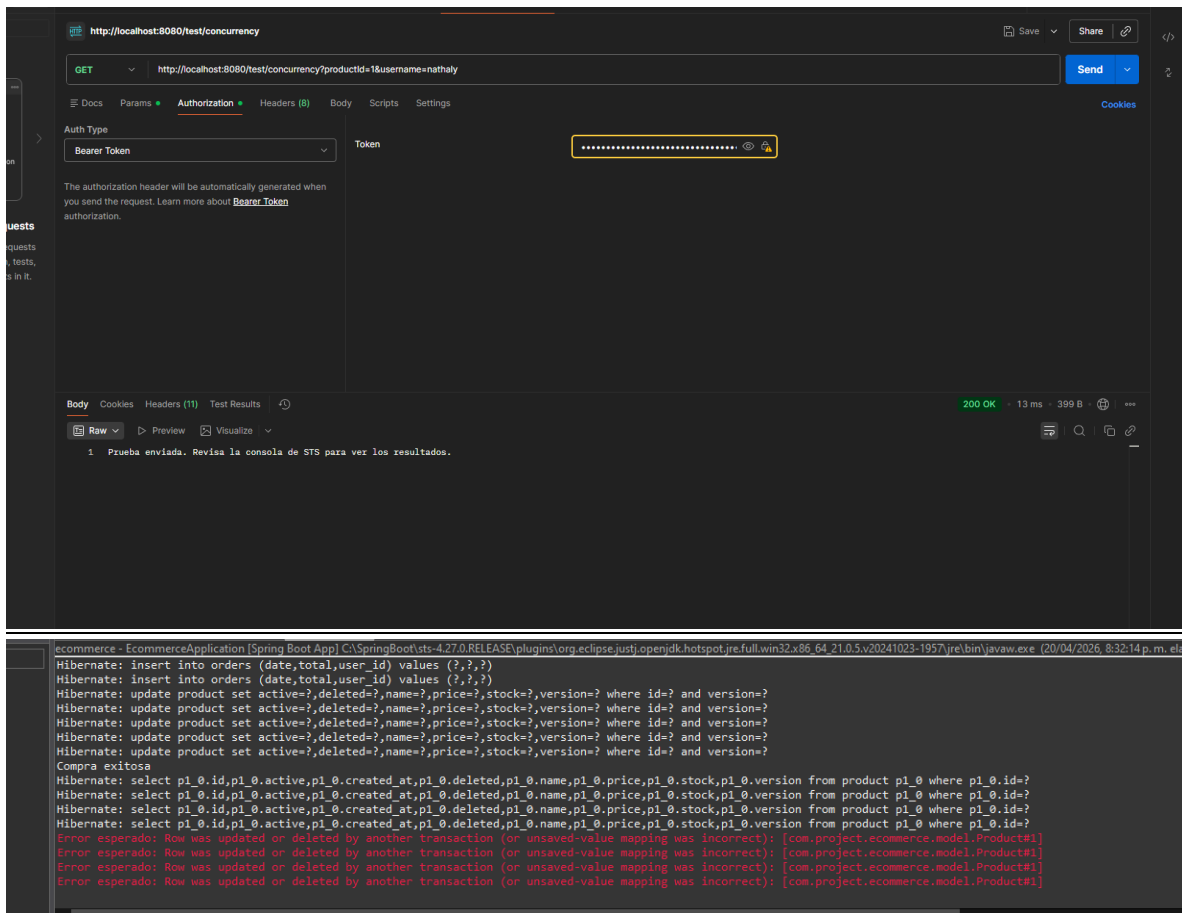
1 package com.project.ecommerce;
2
3 import org.springframework.boot.CommandLineRunner;
10
11 @SpringBootApplication
12 public class EcommerceApplication {
13
14     public static void main(String[] args) {
15         SpringApplication.run(EcommerceApplication.class, args);
16     }
17
18     // Asegúrate de que esto esté dentro de EcommerceApplication
19     @Bean
20     CommandLineRunner initDatabase(UserRepository userRepository) {
21         return args -> {
22             User user = new User();
23             user.setUsername("nathaly");
24             user.setPassword("12345");
25             user.setRole("CLIENTE");
26             userRepository.save(user);
27             System.out.println("USUARIO NATHALY CREADO EXITOSAMENTE");
28         };
29     }
30 }
31
```

```
Problems  Javadoc  Declaration  Console  Terminal
ecommerce - EcommerceApplication (Spring Boot App) C:\SpringBoot\src-4.27.0.RELEASE\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.21.0.5\20241023.1957\jre\bin\java.exe (20/04/2026, 8:32:14 p.m. elapsed: 0:00:39) [pid: 13180]

Maximum pool size: undefined/unknown
2026-04-20T20:32:26.185-05:00 INFO 13180 --- [ecommerce] [ restartedMain] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to
2026-04-20T20:32:26.267-05:00 INFO 13180 --- [ecommerce] [ restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized 3PA EntityManagerFactory for persistence unit 'default'
2026-04-20T20:32:26.551-05:00 WARN 13180 --- [ecommerce] [ restartedMain] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may b
2026-04-20T20:32:26.579-05:00 WARN 13180 --- [ecommerce] [ restartedMain] .s.s.UserDetailsServiceAutoConfiguration :

Using generated security password: 1a910ff1-83f3-47cc-ba5e-d750a5e4fb27

This generated password is for development use only. Your security configuration must be updated before running your application in production.
2026-04-20T20:32:26.586-05:00 INFO 13180 --- [ecommerce] [ restartedMain] *$InitializeUserDetailsServiceConfigurer : Global AuthenticationManager configured with UserDetailsService bean with name s
2026-04-20T20:32:26.940-05:00 INFO 13180 --- [ecommerce] [ restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2026-04-20T20:32:26.969-05:00 INFO 13180 --- [ecommerce] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-04-20T20:32:26.975-05:00 INFO 13180 --- [ecommerce] [ restartedMain] c.p.ecommerce.EcommerceApplication : Started EcommerceApplication in 3.472 seconds (process running for 3.988)
Hibernate: insert into users (active,password,role,username) values (?,?
USUARIO NATHALY CREADO EXITOSAMENTE</pre
```



Persistencia

Base de datos: PostgreSQL (Docker)

ORM: JPA / Hibernate

- Relaciones entre entidades
- Transacciones (@Transactional)
- Soft delete en productos
- Auditoría de eventos

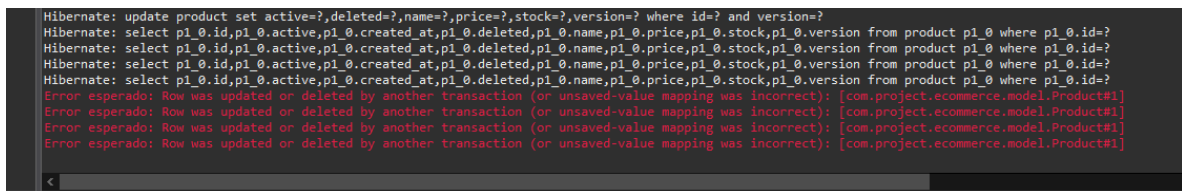
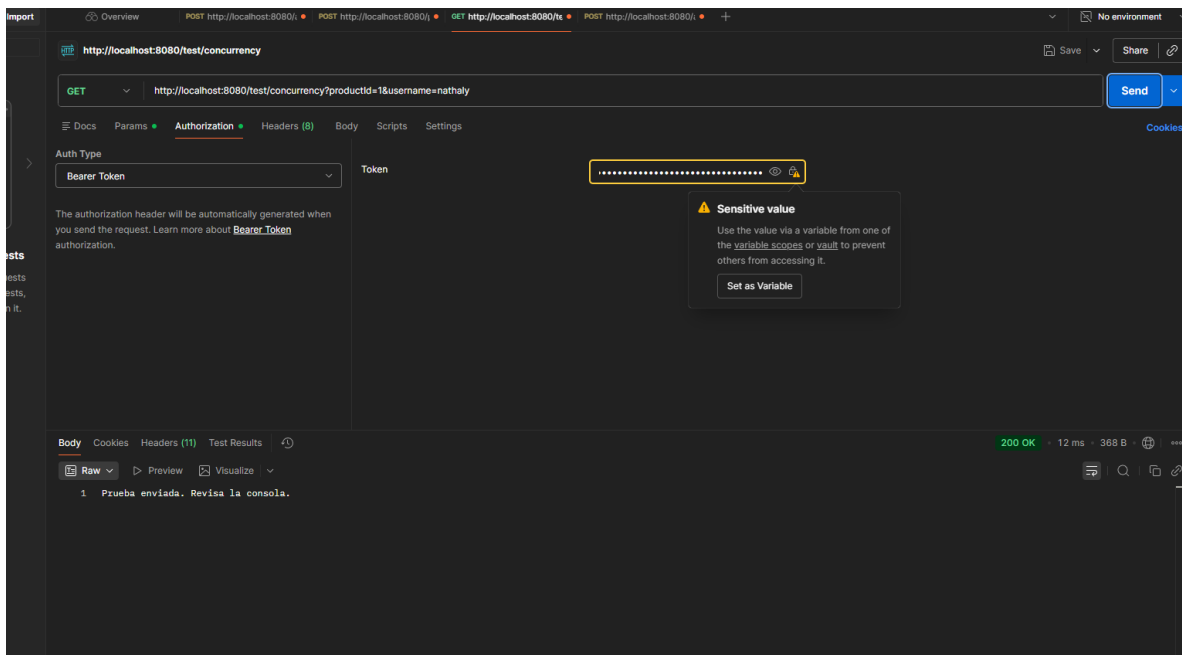
API REST

Endpoints principales:

Auth

auth-controller	
POST	/auth/register
POST	/auth/login

Productos



Use DTOs para evitar exponer entidades directamente y mejorar seguridad también uso de records para simplificar código y hacerlo más moderno el optional para evitar errores de null y los Streams para transformar datos.

Respuestas a preguntas guía

¿Cómo modelaron el dominio?

- Se modeló en base a entidades reales del negocio (usuarios, productos, órdenes), separando responsabilidades y usando relaciones entre tablas.

¿Qué riesgo concurrente identificaron?

- La sobreventa de productos cuando múltiples usuarios compran al mismo tiempo.

¿Cómo lo resolvieron?

- Mediante concurrencia optimista usando `@Version`, evitando conflictos de actualización.

¿Cómo controlan accesos?

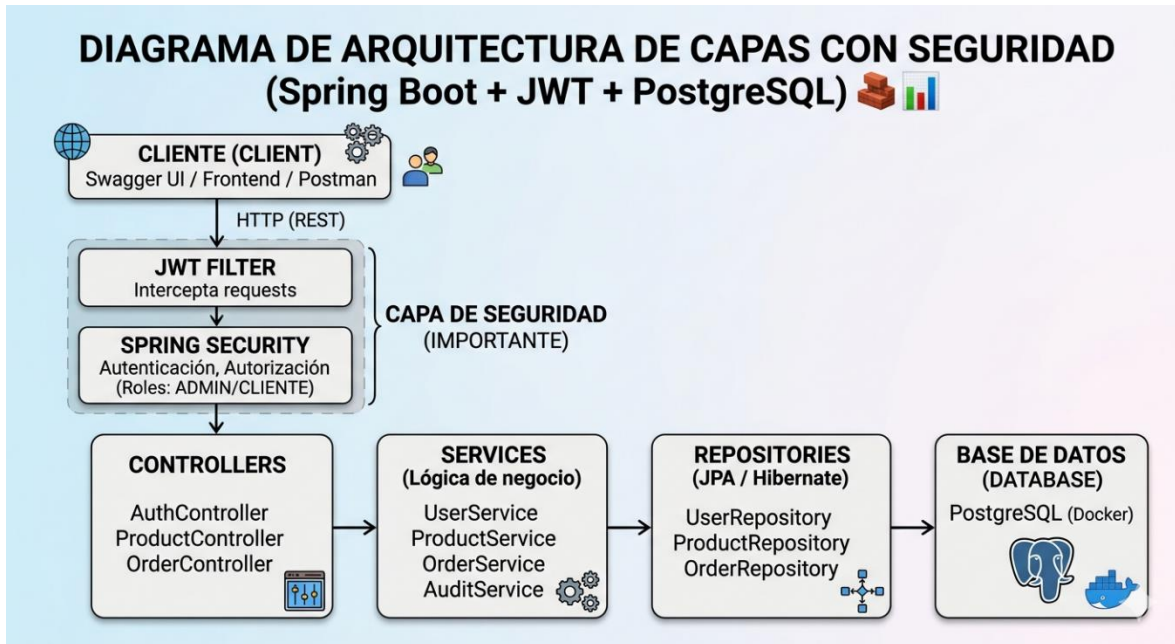
- Con JWT + roles (ADMIN / CLIENTE) y configuración en Spring Security.

¿Por qué arquitectura en capas?

- Porque permite separar responsabilidades, facilitar mantenimiento y escalar el sistema.

Se priorizó concurrencia optimista por su mejor rendimiento en escenarios con baja probabilidad de conflicto.

DIAGRAMA DE ARQUITECTURA (CAPAS)



- El sistema está construido bajo una arquitectura en capas donde cada componente tiene una responsabilidad específica.
- El cliente (Swagger o frontend) envía peticiones HTTP que son interceptadas por Spring Security, donde se valida el token JWT y se verifica el rol del usuario.
- Luego, las solicitudes llegan a los controladores, que reciben y validan los datos de entrada.
- Los controladores delegan la lógica de negocio a los servicios, donde se implementan reglas como validación de stock y creación de órdenes.
- Los servicios interactúan con los repositorios, los cuales se encargan del acceso a la base de datos mediante JPA/Hibernate.
- Finalmente, la información se almacena en PostgreSQL, ejecutándose dentro de un contenedor Docker.

Código y proceso de creación:

ecommerce/pom.xml

docker-compose.yml

docker-compose.yml - The Compose specification establishes a standard for the definition of multi-container platform-agnostic applications (comp

1 version: '3.8'

2

3 services:

4 db:

5 image: postgres:15

6 container_name: ecommerce_db

7 environment:

8 POSTGRES_DB: ecommerce

9 POSTGRES_USER: admin

10 POSTGRES_PASSWORD: admin

11 ports:

12 - "5432:5432"

13

Problems

Javadoc

Declaration

Terminal

C:\Windows\system32\cmd.exe

C:\Users\User\Downloads\ecommerce>docker compose up -d
time="2025-04-19T18:50:22-05:00" level=warning msg="C:\Users\User\Downloads\ecommerce\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 15/15
db Pulled
a8a0725a32a6 Pull complete 17.0s
2c38075ffca2 Pull complete 0.2s
b4765ac2e7e6 Pull complete 15.3s
9328c154beae Pull complete 15.4s
af2a6c5f289c Pull complete 7.1s
5ca81a59b0a4 Pull complete 0.5s
8ecdbd190da6 Pull complete 0.5s
00f33d580dc3 Pull complete 7.4s
c45b346571fc Pull complete 0.6s
71a32c7571f4 Pull complete 0.6s
5c152148754e Pull complete 1.3s
[+] Running 2/2
Network ecommerce_default Created 0.0s
Container ecommerce_db Started 0.7s
C:\Users\User\Downloads\ecommerce>

Writable

Insert

13:1:235

docker desktop PERSONAL

Containers [Give feedback](#) [Learn more](#)

View all your running containers and applications. [Learn more](#)

Container CPU usage 0.00% / 1600% (16 CPUs available)

Container memory usage 23.7MB / 14.95GB

Show charts

Search

Only show running containers

Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
ecommerce				0%	5 minutes ago	

Showing 1 item

Walkthroughs

- Multi-container applications 8 mins
- Containerize your application 3 mins

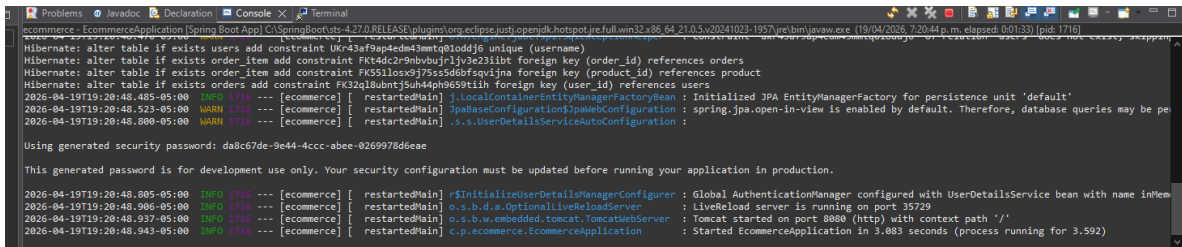
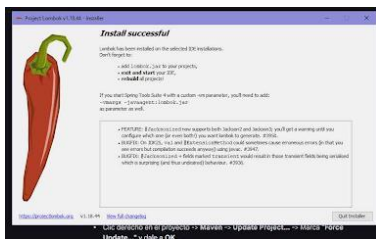
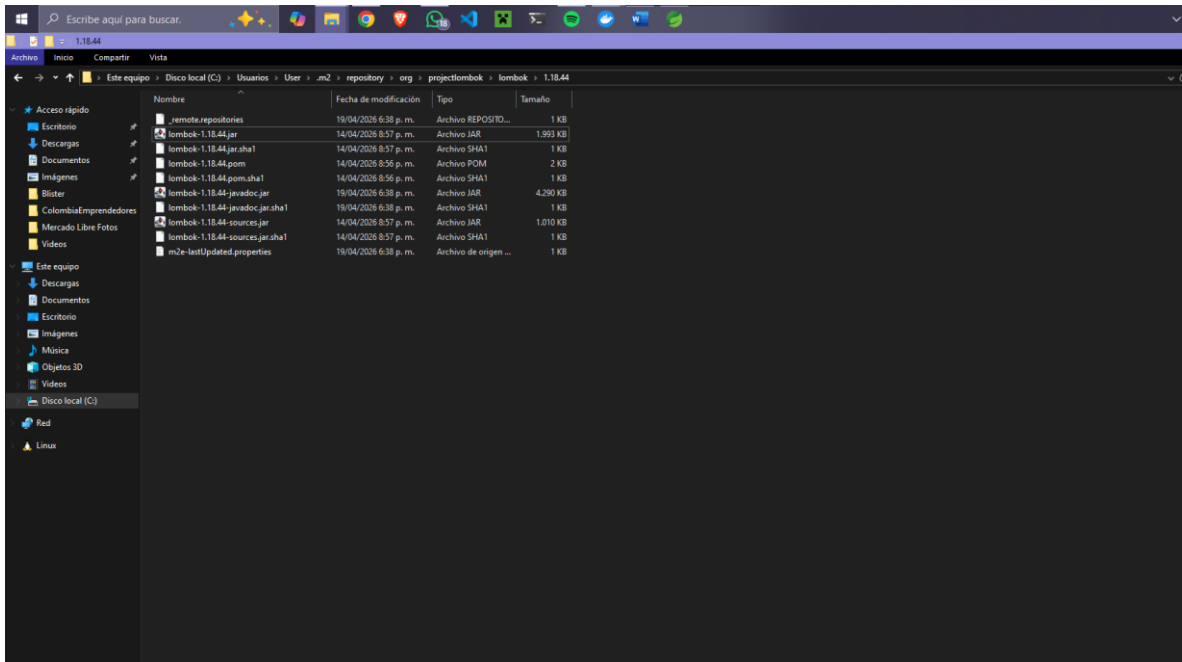
[View more in the Learning center](#)

Engine running RAM 1.04 GB CPU 0.00% Disk 1.71 GB used (limit 1006.85 GB)

Terminal [New version available](#)

ecommerce/pom.xml docker-compose.yml application.properties

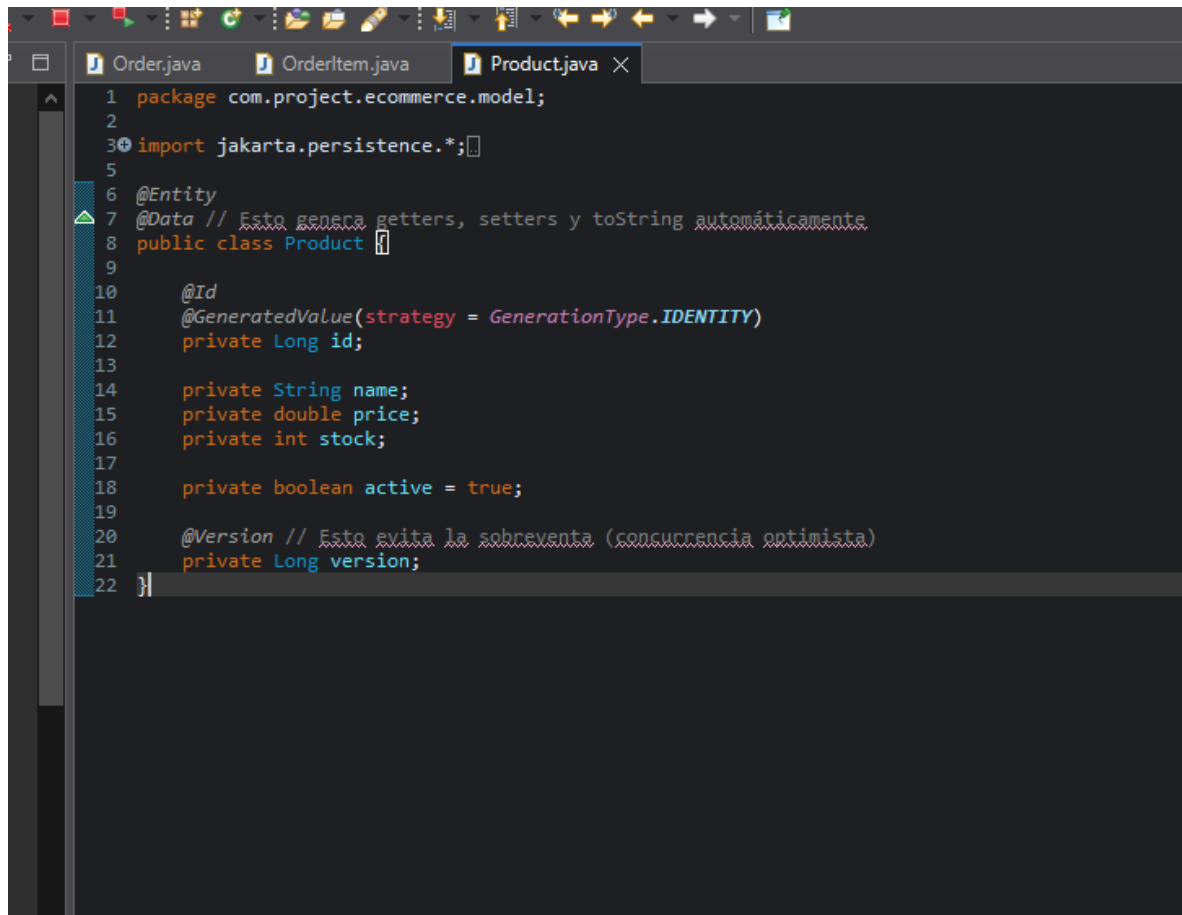
```
1 spring.application.name=ecommerce
2 # Configuración de la Base de Datos (Docker)
3 spring.datasource.url=jdbc:postgresql://localhost:5432/ecommerce
4 spring.datasource.username=admin
5 spring.datasource.password=admin
6 spring.datasource.driver-class-name=org.postgresql.Driver
7
8 # Configuración de JPA / Hibernate
9 spring.jpa.hibernate.ddl-auto=update
10 spring.jpa.show-sql=true
11 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
12
13 # Puerto del servidor
14 server.port=8080
15
```



Model

```
1 Order.java X
2
3 package com.project.ecommerce.model;
4
5 import jakarta.persistence.*;
6
7 @Entity
8 @Table(name = "orders")
9 @Data
10 public class Order {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long id;
15
16     @ManyToOne
17     @JoinColumn(name = "user_id")
18     private User user;
19
20     private LocalDateTime date = LocalDateTime.now();
21
22     private double total;
23
24     @OneToMany(mappedBy = "order", cascade = CascadeType.ALL)
25     private List<OrderItem> items;
26 }
27
28
```

```
1 OrderItem.java X
2
3 package com.project.ecommerce.model;
4
5 import jakarta.persistence.*;
6
7 @Entity
8 @Data
9 public class OrderItem {
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @ManyToOne
16     @JoinColumn(name = "order_id")
17     private Order order;
18
19     @ManyToOne
20     @JoinColumn(name = "product_id")
21     private Product product;
22
23     private int quantity;
24     private double price; // Precio al momento de la compra
25 }
26
```



```
1 package com.project.ecommerce.model;
2
3 import jakarta.persistence.*;
4
5
6 @Entity
7 @Data // Esto genera getters, setters y toString automáticamente
8 public class Product {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Long id;
13
14     private String name;
15     private double price;
16     private int stock;
17
18     private boolean active = true;
19
20     @Version // Esto evita la sobreventa (concurrency optimista)
21     private Long version;
22 }
```

```
Order.java  OrderItem.java  Product.java  User.java X
1 package com.project.ecommerce.model;
2
3 import jakarta.persistence.*;
4
5
6 @Entity
7 @Table(name = "users")
8 @Data
9 public class User {
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @Column(unique = true)
16     private String username;
17     private String password;
18     private String role;
19
20     private boolean active = true;
21 }
22
```

Repository

```
Order.java  OrderItem.java  Product.java  User.java  OrderItemRepository.java
1 package com.project.ecommerce.repository;
2 import com.project.ecommerce.model.OrderItem;
3
4
5 public interface OrderItemRepository extends JpaRepository<OrderItem, Long> {
6 }
7
```



```
Order.java  OrderItem.java  Product.java  User.java  OrderItemRepository.java  OrderRepository.java X
1 package com.project.ecommerce.repository;
2 import com.project.ecommerce.model.Order;
3
4
5
6
7 public interface OrderRepository extends JpaRepository<Order, Long> {
8     List<Order> findByUser(User user);
9 }
10
```

```
Order.java  OrderItem.java  Product.java  User.java  OrderItemRepository.java  OrderRepository.java  ProductRepository.java X
1 package com.project.ecommerce.repository;
2 import com.project.ecommerce.model.Product;
3
4
5 public interface ProductRepository extends JpaRepository<Product, Long> {
6 }
7
```

```
Order.java  OrderItem.java  Product.java  User.java  OrderItemRepository.java  OrderRepository.java  ProductRepository.java  UserRepository.java X
1 package com.project.ecommerce.repository;
2 import com.project.ecommerce.model.User;
3
4
5
6 public interface UserRepository extends JpaRepository<User, Long> {
7     Optional<User> findByUsername(String username);
8 }
9
```

Controller

```
1 package com.project.ecommerce.controller;
2
3 import com.project.ecommerce.dto.ProductRequest;
4
5
6
7
8
9
10 @RestController
11 @RequestMapping("/products")
12 public class ProductController {
13
14     @Autowired
15     private ProductService service;
16
17     // Endpoint para crear productos
18     @PostMapping
19     public ResponseEntity<ProductResponse> create(@RequestBody ProductRequest request) {
20         return ResponseEntity.ok(service.create(request));
21     }
22
23     // Endpoint para comprar (simular concurrencia)
24     @PostMapping("/{id}/buy")
25     public ResponseEntity<String> buy(@PathVariable Long id, @RequestParam int quantity) {
26         service.buyProduct(id, quantity);
27         return ResponseEntity.ok("Compra realizada con éxito");
28     }
29
30     @GetMapping
31     public ResponseEntity<?> listAll() {
32         return ResponseEntity.ok(service.findAll());
33     }
34
35 }
36
```

DTO

```
1 package com.project.ecommerce.dto;
2
3 public record ProductRequest(String name, double price, int stock) {}
4
```

```
1 package com.project.ecommerce.dto;
2
3 public record ProductResponse(Long id, String name, double price, int stock) {}
4
```

com.project.ecommerce.dto

- ProductResponse
 - id: Long
 - name: String
 - price: double
 - stock: int

```
1 package com.project.ecommerce.exception;
2
3 import org.springframework.http.ResponseEntity;
4
5 @RestControllerAdvice
6 public class GlobalExceptionHandler {
7
8     // Maneja errores de negocio (como "No hay stock")
9     @ExceptionHandler(RuntimeException.class)
10     public ResponseEntity<String> handleRuntime(RuntimeException ex) {
11         return ResponseEntity.badRequest().body(ex.getMessage());
12     }
13
14     // Captura el error de concurrencia cuando dos personas compran al mismo tiempo
15     @ExceptionHandler(ObjectOptimisticLockingFailureException.class)
16     public ResponseEntity<String> handleConcurrency(ObjectOptimisticLockingFailureException ex) {
17         return ResponseEntity.status(409).body("Error de concurrencia: El producto fue actualizado por otro usuario. Intenta de nuevo.");
18     }
19 }
20
21
22
23
```

```
1 package com.project.ecommerce.service;
2
3 import com.project.ecommerce.model.Product;
4
5 @Service
6 public class ProductService {
7
8     @Autowired
9     private ProductRepository productRepository;
10
11     @Transactional
12     public ProductResponse create(ProductRequest request) {
13         Product product = new Product();
14         product.setName(request.name());
15         product.setPrice(request.price());
16         product.setStock(request.stock());
17
18         Product saved = productRepository.save(product);
19         return new ProductResponse(saved.getId(), saved.getName(), saved.getPrice(), saved.getStock());
20     }
21
22     @Transactional
23     public void buyProduct(long productId, int quantity) {
24         // Buscamos el producto
25         Product product = productRepository.findById(productId)
26             .orElseThrow(() -> new RuntimeException("Producto no encontrado"));
27
28         // Validamos stock
29         if (product.getStock() < quantity) {
30             throw new RuntimeException("No hay stock suficiente para " + product.getName());
31         }
32
33         // Restamos stock
34         product.setStock(product.getStock() - quantity);
35
36         // Guardamos. Gracias al @Version en el Model, si alguien más
37         // compra el mismo producto al mismo tiempo, esta lanzará una excepción
38         // evitando la sobreventa.
39     }
40 }
41
42
43
44
45
46
47
```

```
Commerce - EcommerceApplication [Spring Boot App] C:\SpringBoot\sts-4.27.0.RELEASE\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.21.0.5.v20241023-1957\jre\bin\java.exe (19/04/2026, 7:20:44 p.m. elapsed: 0:41:32)
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
26-04-19T20:01:37.513-05:00 INFO 1716 --- [ecommerce] [ restartedMain] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA platform available (set 'hibernate.transaction
26-04-19T20:01:37.541-05:00 INFO 1716 --- [ecommerce] [ restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'de
26-04-19T20:01:37.582-05:00 WARN 1716 --- [ecommerce] [ restartedMain] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, dat
26-04-19T20:01:37.690-05:00 WARN 1716 --- [ecommerce] [ restartedMain] .s.s.UserDetailsServiceAutoConfiguration :

ng generated security password: 365ad7f0-903a-428a-881d-bb773f31644f

s generated password is for development use only. Your security configuration must be updated before running your application in production.
26-04-19T20:01:37.691-05:00 INFO 1716 --- [ecommerce] [ restartedMain] r$InitializeUserDetailsServiceConfigurer : Global AuthenticationManager configured with UserDetailsService
26-04-19T20:01:37.713-05:00 INFO 1716 --- [ecommerce] [ restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
26-04-19T20:01:37.723-05:00 INFO 1716 --- [ecommerce] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
26-04-19T20:01:37.725-05:00 INFO 1716 --- [ecommerce] [ restartedMain] c.p.ecommerce.EcommerceApplication : Started EcommerceApplication in 0.572 seconds (process running
26-04-19T20:01:37.726-05:00 INFO 1716 --- [ecommerce] [ restartedMain] .ConditionEvaluationDeltaLoggingListener : Condition evaluation unchanged

Writable Smart Insert 46: 80: 1681
```

localhost:8080/login

Tructor de Google Redes Universidad programación cursos Trabajo Mercado Libre Colo... Flex | Configuración Softaculous - Softac... Mi página

Please sign in

user

Password

Sign in

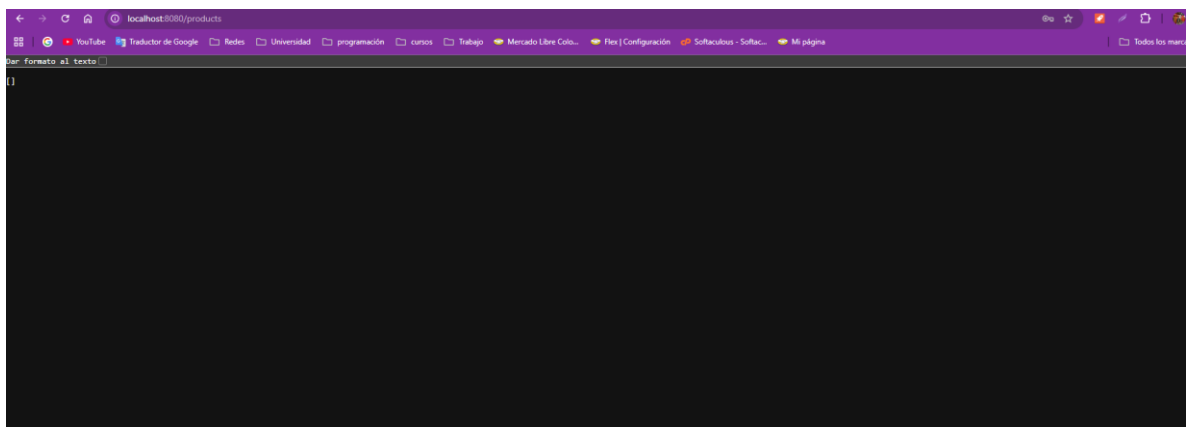
Using generated security password: 365ad7f0-903a-428a-881d-bb773f31644f

Please sign in

user

.....

Sign in



Security

```
1 package com.project.ecommerce.security;
2
3 import jakarta.servlet.FilterChain;
4 import jakarta.servlet.ServletException;
5 import jakarta.servlet.http.HttpServletRequest;
6 import jakarta.servlet.http.HttpServletResponse;
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
9 import org.springframework.security.core.context.SecurityContextHolder;
10 import org.springframework.stereotype.Component;
11 import org.springframework.web.filter.OncePerRequestFilter;
12 import java.io.IOException;
13 import java.util.ArrayList;
14 import com.project.ecommerce.security.JwtUtil;
15
16 @Component
17 public class JwtFilter extends OncePerRequestFilter {
18
19     @Autowired
20     private JwtUtil jwtUtil;
21
22     @Override
23     protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)
24         throws ServletException, IOException {
25
26         String authHeader = request.getHeader("Authorization");
27
28         if (authHeader != null && authHeader.startsWith("Bearer ")) {
29             String token = authHeader.substring(7);
30             if (jwtUtil.isTokenValid(token)) {
31                 String username = jwtUtil.extractUsername(token);
32                 UsernamePasswordAuthenticationToken auth = new UsernamePasswordAuthenticationToken(username, null, new ArrayList<>());
33                 SecurityContextHolder.getContext().setAuthentication(auth);
34             }
35         }
36         filterChain.doFilter(request, response);
37     }
38 }
```

```
8 import java.security.Key;
9 import java.util.Date;
10
11 @Component
12 public class JwtUtil {
13
14     // Generamos una llave secreta de forma más directa
15     private static final Key KEY = Keys.secretKeyFor(SignatureAlgorithm.HS256);
16     private final int expirationMillis = 3600000; // 1 hora
17
18     public String generateToken(String username) {
19         return Jwts.builder()
20             .setSubject(username)
21             .setIssuedAt(new Date())
22             .setExpiration(new Date(System.currentTimeMillis() + expirationMillis))
23             .signWith(KEY)
24             .compact();
25     }
26
27     public String extractUsername(String token) {
28         return Jwts.parserBuilder()
29             .setSigningKey(KEY)
30             .build()
31             .parseClaimsJws(token)
32             .getBody()
33             .getSubject();
34     }
35
36     public boolean isTokenValid(String token) {
37         try {
38             Jwts.parserBuilder().setSigningKey(KEY).build().parseClaimsJws(token);
39             return true;
40         } catch (Exception e) {
41             return false;
42         }
43     }
44 }
45
```

```
1 package com.project.ecommerce.security;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
6 import org.springframework.security.config.http.SessionCreationPolicy;
7 import org.springframework.security.web.SecurityFilterChain;
8 import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
9
10 @Configuration
11 public class SecurityConfig {
12
13     private final JwtFilter jwtFilter;
14
15     public SecurityConfig(JwtFilter jwtFilter) {
16         this.jwtFilter = jwtFilter;
17     }
18
19     @Bean
20     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
21         return http
22             .csrf(csrf -> csrf.disable()) // Desactivamos CSRF para APIs
23             .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
24             .authorizeHttpRequests(auth -> auth
25                 .requestMatchers("/auth/**").permitAll() // Permite el login
26                 .anyRequest().authenticated()
27             )
28             .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
29             .build();
30     }
31 }
32
```

```
ecommerce - EcommerceApplication [Spring Boot App] C:\Spring\boot\sts-4.27.0.RELEASE\plugins\org.eclipse.jdt.openjdk.hotspot.jre.full.win32.x86_64.21.0.5.v20241023-1957\jre\bin\java.exe (19/04/2026, 8:32:44 p.m. elapsed: 0:09:00) [pid: 14900]
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-04-19T20:34:18.181-05:00 INFO 14900 --- [ecommerce] [ restartedMain] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to ei
2026-04-19T20:34:18.212-05:00 INFO 14900 --- [ecommerce] [ restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-04-19T20:34:18.255-05:00 WARN 14900 --- [ecommerce] [ restartedMain] jpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be p
2026-04-19T20:34:18.269-05:00 WARN 14900 --- [ecommerce] [ restartedMain] .s.UserDetailsServiceAutoConfiguration :

Using generated security password: 90c10572-f8c4-4b8a-96ce-41d4b24cfdbb

This generated password is for development use only. Your security configuration must be updated before running your application in production.
2026-04-19T20:34:18.261-05:00 INFO 14900 --- [ecommerce] [ restartedMain] r$InitializerUserDetailsServiceConfigurer : Global AuthenticationManager configured with UserDetailsService bean with name inMe
2026-04-19T20:34:18.358-05:00 INFO 14900 --- [ecommerce] [ restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2026-04-19T20:34:18.358-05:00 INFO 14900 --- [ecommerce] [ restartedMain] o.s.b.w.e.t.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-04-19T20:34:18.361-05:00 INFO 14900 --- [ecommerce] [ restartedMain] c.p.ecommerce.EcommerceApplication : Started EcommerceApplication in 0.552 seconds (process running for 93.566)
2026-04-19T20:34:18.362-05:00 INFO 14900 --- [ecommerce] [ restartedMain] .ConditionEvaluationLoggingListener : Condition evaluation unchanged
```

Controller

```
1 package com.project.ecommerce.controller;
2
3 import com.project.ecommerce.security.JwtUtil;
4 import org.springframework.beans.factory.annotation.Autowired;
5 import org.springframework.web.bind.annotation.*;
6
7 import java.util.Map;
8
9 @RestController
10 @RequestMapping("/auth")
11 public class AuthController {
12
13     @Autowired
14     private JwtUtil jwtUtil;
15
16     // Endpoint para obtener el token
17     @PostMapping("/login")
18     public Map<String, String> login(@RequestBody Map<String, String> request) {
19         String username = request.get("username");
20         // Aquí simplifiquemos: si existe cualquier usuario, le damos token
21         String token = jwtUtil.generateToken(username);
22         return Map.of("token", token);
23     }
24 }
25
```

```
ecommerce - EcommerceApplication [Spring Boot App] C:\Spring\boot\sts-4.27.0.RELEASE\plugins\org.eclipse.jdt.openjdk.hotspot.jre.full.win32.x86_64.21.0.5.v20241023-1957\jre\bin\java.exe (19/04/2026, 8:32:44 p.m. elapsed: 0:09:44) [pid: 14900]
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-04-19T20:42:28.568-05:00 INFO 14900 --- [ecommerce] [ restartedMain] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to ei
2026-04-19T20:42:28.597-05:00 INFO 14900 --- [ecommerce] [ restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-04-19T20:42:28.637-05:00 WARN 14900 --- [ecommerce] [ restartedMain] jpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be p
2026-04-19T20:42:28.642-05:00 WARN 14900 --- [ecommerce] [ restartedMain] .s.UserDetailsServiceAutoConfiguration :

Using generated security password: 9339198f-0034-4de4-be82-f9479367dbc5

This generated password is for development use only. Your security configuration must be updated before running your application in production.
2026-04-19T20:42:28.642-05:00 INFO 14900 --- [ecommerce] [ restartedMain] r$InitializerUserDetailsServiceConfigurer : Global AuthenticationManager configured with UserDetailsService bean with name inMe
2026-04-19T20:42:28.736-05:00 INFO 14900 --- [ecommerce] [ restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2026-04-19T20:42:28.744-05:00 INFO 14900 --- [ecommerce] [ restartedMain] o.s.b.w.e.t.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-04-19T20:42:28.746-05:00 INFO 14900 --- [ecommerce] [ restartedMain] c.p.ecommerce.EcommerceApplication : Started EcommerceApplication in 0.523 seconds (process running for 583.952)
2026-04-19T20:42:28.747-05:00 INFO 14900 --- [ecommerce] [ restartedMain] .ConditionEvaluationLoggingListener : Condition evaluation unchanged
```

Prueba

Product.javaProductService.javaJwtFilter.javaJwtUtil.javaecommerce/p...SecurityConf...AuthControll...OrderService...

5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.stereotype.Service;
7 import org.springframework.transaction.annotation.Transactional;
8 import java.util.List;
9
10 @Service
11 public class OrderService {
12
13 @Autowired
14 private OrderRepository orderRepository;
15 @Autowired
16 private ProductRepository productRepository;
17 @Autowired
18 private UserRepository userRepository;
19
20 @Transactional
21 public Order placeOrder(String username, Long productId, int quantity) {
22 // 1. Buscar usuario y producto
23 User user = userRepository.findByUsername(username)
24 .orElseThrow(() -> new RuntimeException("Usuario no encontrado"));
25 Product product = productRepository.findById(productId)
26 .orElseThrow(() -> new RuntimeException("Producto no encontrado"));
27
28 // 2. Validar Stock (Bodega de negocio central)
29 if (product.getStock() < quantity) {
30 throw new RuntimeException("Stock insuficiente para: " + product.getName());
31 }
32
33 // 3. Disminuir stock
34 product.setStock(product.getStock() - quantity);
35 productRepository.save(product);
36
37 // 4. Crear la orden
38 Order order = new Order();
39 order.setUser(user);
40 order.setTotal(product.getPrice() * quantity);
41
42 return orderRepository.save(order);
43 }
44 }
45

OutlineTask List
com.project.ecommerce
OrderService

ProblemsJavadocDeclarationConsoleTerminal

ecommerce - EcommerceApplication [Spring Boot App] C:\SpringBoot\sts-4.27.0.RELEASE\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.21.0.5.v20241023-1957\jre\bin\javaw.exe (19/04/2026, 8:32:44 p.m. elapsed: 0:11:14)
Minimum pool size: undefined/unknown
2026-04-19T20:43:54.237-05:00 INFO 14900 --- [ecommerce] [restartedMain] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA platform available (set 'hibernate.transac
2026-04-19T20:43:54.266-05:00 INFO 14900 --- [ecommerce] [restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'd
2026-04-19T20:43:54.321-05:00 WARN 14900 --- [ecommerce] [restartedMain] JpaBaseConfiguration\$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, da
2026-04-19T20:43:54.326-05:00 WARN 14900 --- [ecommerce] [restartedMain] .s.s.UserDetailsServiceAutoConfiguration :

Using generated security password: cff688d0-77e2-467b-920d-732c71a74190

This generated password is for development use only. Your security configuration must be updated before running your application in production.
2026-04-19T20:43:54.326-05:00 INFO 14900 --- [ecommerce] [restartedMain] r.InitializeUserDetailsServiceConfigurer : Global AuthenticationManager configured with UserDetailsService
2026-04-19T20:43:54.395-05:00 INFO 14900 --- [ecommerce] [restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2026-04-19T20:43:54.404-05:00 INFO 14900 --- [ecommerce] [restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-04-19T20:43:54.408-05:00 INFO 14900 --- [ecommerce] [restartedMain] c.p.ecommerce.EcommerceApplication : Started EcommerceApplication in 0.512 seconds (process running for 0.512 seconds)
2026-04-19T20:43:54.409-05:00 INFO 14900 --- [ecommerce] [restartedMain] .ConditionEvaluationDeltaLoggingListener : Condition evaluation unchanged

WritableSmart Insert45: 1: 1551


```
erjava - Spring Tools for Eclipse

1 package com.project.ecommerce.controller;
2
3 import com.project.ecommerce.service.OrderService;
4 import org.springframework.beans.factory.annotation.Autowired;
5 import org.springframework.web.bind.annotation.*;
6 import java.util.concurrent.ExecutorService;
7 import java.util.concurrent.Executors;
8
9 @RestController
10 @RequestMapping("/test")
11 public class TestController {
12
13     @Autowired
14     private OrderService orderService;
15
16     @GetMapping("/concurrency")
17     public String testConcurrency(@RequestParam Long productId, @RequestParam String username) {
18         ExecutorService executor = Executors.newFixedThreadPool(5);
19
20         for (int i = 0; i < 5; i++) {
21             executor.submit(() -> {
22                 try {
23                     orderService.placeOrder(username, productId, 1);
24                     System.out.println("Compra exitosa");
25                 } catch (Exception e) {
26                     System.err.println("Error esperado: " + e.getMessage());
27                 }
28             });
29         }
30         return "Prueba enviada. Revisa la consola de STS para ver los resultados.";
31     }
32 }
33
```

com.project.ecommerce.controller
TestController
orderService: OrderService
testConcurrency(Long, String): String

ecommerce - EcommerceApplication [Spring Boot App] C:\Spring\Boot\sts-4.27.0.RELEASE\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.5.v20241023-1957\jre\bin\javaw.exe (19/04/2026 8:32:44 p. m. elapsed: 0:12:07) [pid: 14900]

2026-04-19T20:44:37.044-05:00 INFO 14900 --- [ecommerce] [restartedMain] o.h.e.t.j.p.l.JtaPlatformInitiator : HHH0000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to e

2026-04-19T20:44:37.072-05:00 INFO 14900 --- [ecommerce] [restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'

2026-04-19T20:44:37.128-05:00 WARN 14900 --- [ecommerce] [restartedMain] JpaBaseConfigurationJpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be p

2026-04-19T20:44:37.133-05:00 WARN 14900 --- [ecommerce] [restartedMain] .s.s.UserDetailsServiceAutoConfiguration :

Using generated security password: fa786ccd-f9fa-4d21-ad3a-bdd9572f91d

This generated password is for development use only. Your security configuration must be updated before running your application in production.

2026-04-19T20:44:37.133-05:00 INFO 14900 --- [ecommerce] [restartedMain] r\$InitializeUserDetailsServiceConfigurer : Global AuthenticationManager configured with UserDetailsService bean with name inMe

2026-04-19T20:44:37.201-05:00 INFO 14900 --- [ecommerce] [restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729

2026-04-19T20:44:37.209-05:00 INFO 14900 --- [ecommerce] [restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'

2026-04-19T20:44:37.211-05:00 INFO 14900 --- [ecommerce] [restartedMain] c.p.ecommerce.EcommerceApplication : Started EcommerceApplication in 0.507 seconds (process running for 712.417)

2026-04-19T20:44:37.212-05:00 INFO 14900 --- [ecommerce] [restartedMain] .ConditionEvaluationDeltaLoggingListener : Condition evaluation unchanged

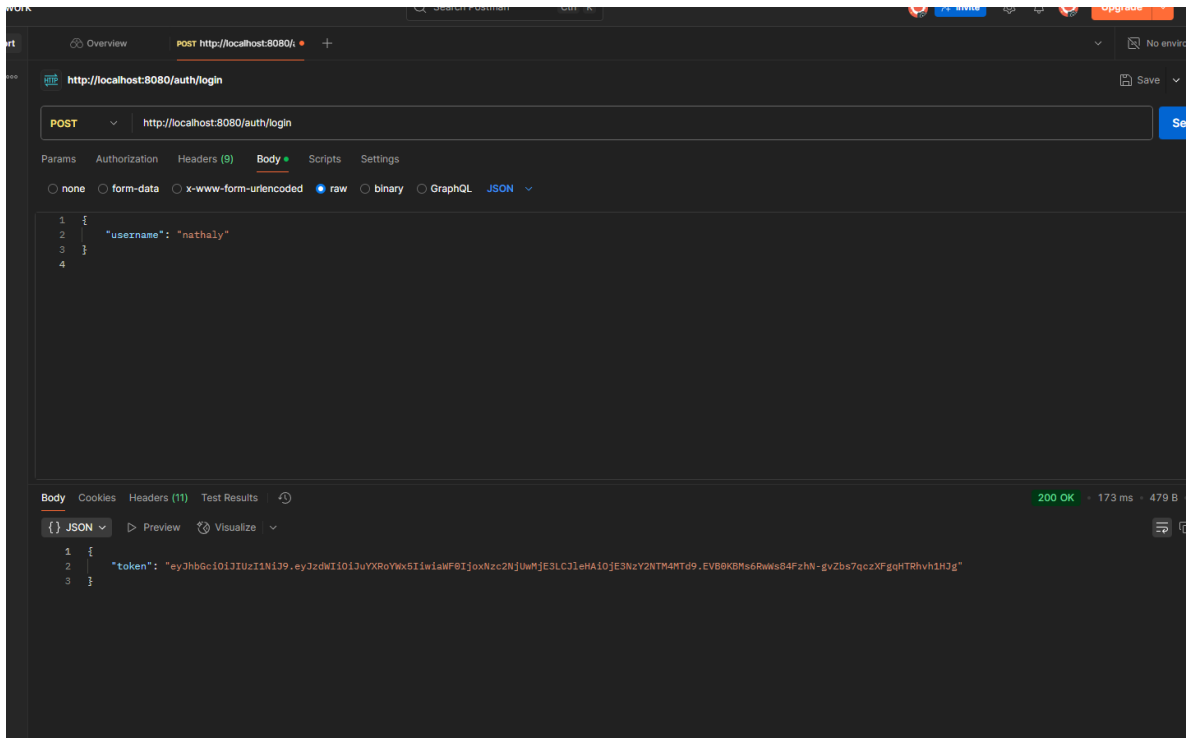
Postman

```
Overview POST http://localhost:8080/ + No en

http://localhost:8080/auth/login Save

POST http://localhost:8080/auth/login

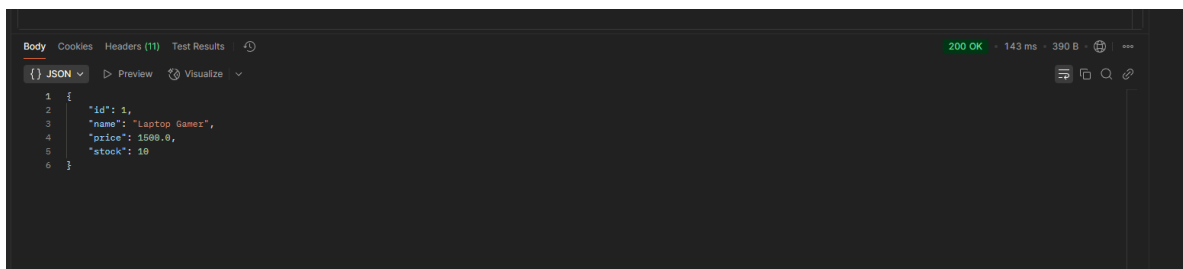
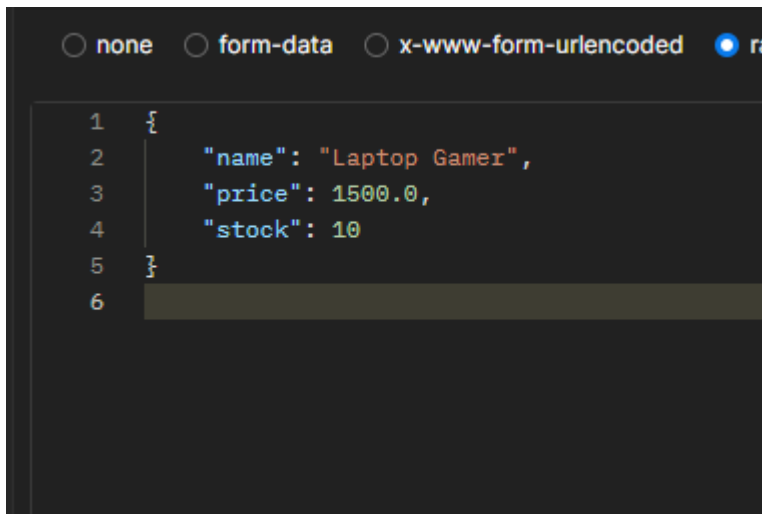
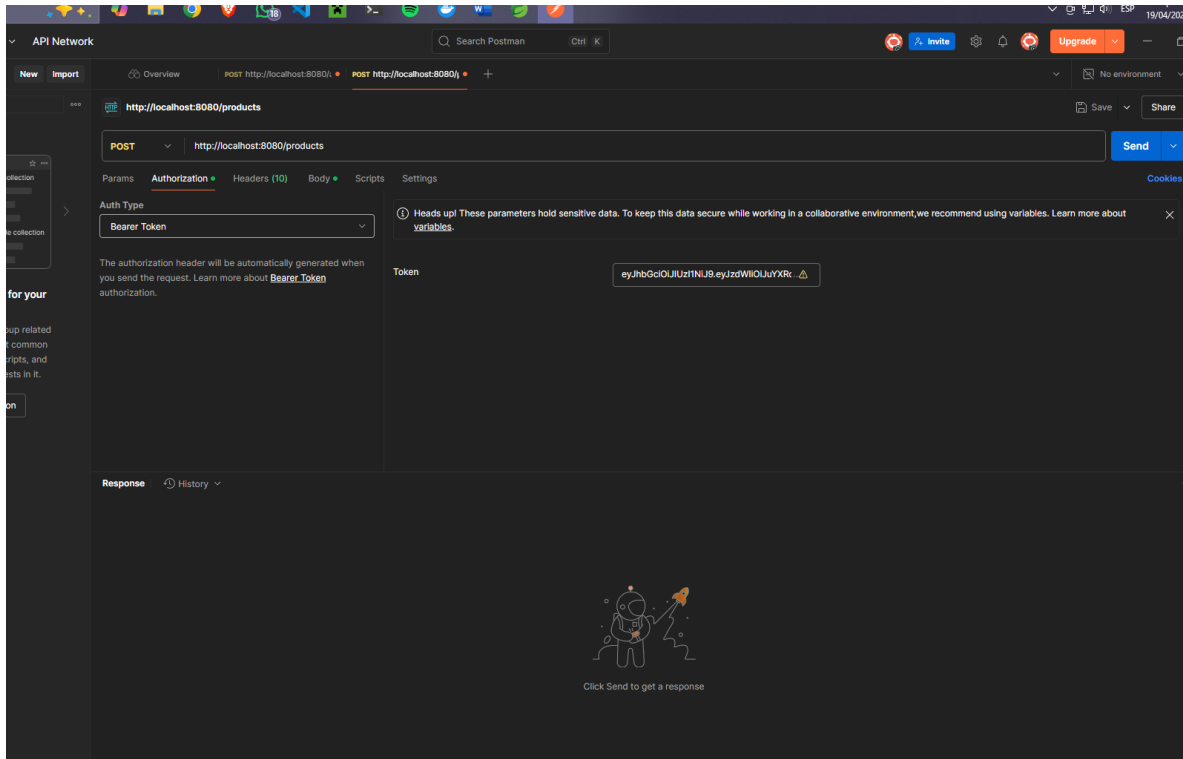
Params Authorization Headers (9) Body Scripts Settings
none form-data x-www-form-urlencoded raw binary GraphQL JSON
1 {
2   | "username": "nathaly"
3   |
4   |
Response History
```



{

"token":
"eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJuYXRoYXN5IiwiaWF0IjoxNzc2NjUwMjE3LCJleHAiOiE3NzY2NTM4MTd9.EVB0KBM6Rws84FzhN-gvZbs7qczXFgqHTRhvh1HJg"

}



Prueba

Overview POST http://localhost:8080/ GET http://localhost:8080/test/concurrency

GET http://localhost:8080/test/concurrency?productId=1&username=nathaly

Docs Params Authorization Headers (8) Body Scripts Settings

Auth Type: Bearer Token

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Token: [REDACTED]

Body Cookies Headers (11) Test Results

Raw Preview Visualize

1 Prueba enviada. Revisa la consola de STS para ver los resultados.

```
2026-04-20T20:13:31.434-05:00 INFO 19544 --- [ecommerce] [ restartedMain] c.p.ecommerce.EcommerceApplication : Started EcommerceAppli
2026-04-20T20:13:35.454-05:00 INFO 19544 --- [ecommerce] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring Di
2026-04-20T20:13:35.454-05:00 INFO 19544 --- [ecommerce] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet '
2026-04-20T20:13:35.455-05:00 INFO 19544 --- [ecommerce] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initializati
Hibernate: insert into product (active,created_at,deleted,name,price,stock,version) values (?,?,?,?,?,?)
Hibernate: select u1_0.id,u1_0.active,u1_0.password,u1_0.role,u1_0.username from users u1_0 where u1_0.username=?
Hibernate: select u1_0.id,u1_0.active,u1_0.password,u1_0.role,u1_0.username from users u1_0 where u1_0.username=?
Hibernate: select u1_0.id,u1_0.active,u1_0.password,u1_0.role,u1_0.username from users u1_0 where u1_0.username=?
Hibernate: select u1_0.id,u1_0.active,u1_0.password,u1_0.role,u1_0.username from users u1_0 where u1_0.username=?
Error esperado: Usuario no encontrado
Error esperado: Usuario no encontrado
Error esperado: Usuario no encontrado
Error esperado: Usuario no encontrado
Error esperado: Usuario no encontrado
```